C++ STL Cheat Sheet — for DSA

Standard Template Library quick reference · Pair · Containers · Iterators · Algorithms · Time Complexities

1. What is STL?

A library of pre-built, template-based (generic) data structures and functions. Four parts:

- **Containers** — store data (vector, set, map...)
- **Algorithms** — operate on containers (sort, find...)
- **Iterators** — "pointers" to traverse containers
- **Functors** — objects usable like functions

```
#include <bits/stdc++.h>
using namespace std;
```

^ includes almost everything — standard for CP/DSA.

2. Pair

O(1) access

```
pair<int, string> p = {1, "abc"};
p.first;  p.second;

// nested pair
pair<int, pair<int,int>> nested = {1,{2,3}};
nested.second.first;

// array of pairs
pair<int,int> arr[] = {{1,2},{3,4}};
```

3. Vector

dynamic array

```
vector<int> v;                // empty
vector<int> v(5);             // size 5, all 0
vector<int> v(5, 100);        // size 5, all 100
vector<int> v2(v1);           // copy
vector<vector<int>> grid(n, vector<int>(m,0)); // 2D
```

Function	Use / Complexity
push_back(x)	insert at end — O(1) amortized
pop_back()	remove last — O(1)
size()	no. of elements — O(1)
v[i] / at(i)	random access — O(1)
front() / back()	first / last element
begin()/end()	iterators (end is 1 past last)
rbegin()/rend()	reverse iterators
clear()	remove all — O(n)
empty()	true if size==0
insert(it, x)	insert x before it — O(n)
erase(it)	remove — O(n)

```
for (int x : v) cout << x;            // range-based
for (auto it = v.begin(); it != v.end(); it++) cout << *it;
sort(v.begin(), v.end());             // ascending
sort(v.begin(), v.end(), greater<int>()); // descending
```

4. List & Deque

list = doubly linked list. **deque** = double-ended queue (array-like, fast both ends).

```
list<int> l;  deque<int> dq;
// both support:
push_back(x); push_front(x);   // O(1)
pop_back();   pop_front();     // O(1)
front(); back(); size(); empty();
```

deque additionally supports `dq[i]` random access in O(1) (list does not).

5. Stack & Queue

adapters

```
stack<int> st;
st.push(x); st.pop(); st.top();
st.empty(); st.size();          // LIFO, all O(1)

queue<int> q;
q.push(x); q.pop(); q.front(); q.back();
q.empty(); q.size();            // FIFO, all O(1)
```

6. Priority Queue

heap

```
priority_queue<int> pq;              // max-heap (default)
pq.push(x); pq.pop(); pq.top();      // O(log n) push/pop

// min-heap:
priority_queue<int, vector<int>, greater<int>> minpq;
```

top() = O(1). Used heavily for "kth largest/smallest", Dijkstra, merging.

7. Set family

sorted, unique*

Container	Property
set<T>	sorted, unique — O(log n) ops
multiset<T>	sorted, duplicates allowed
unordered_set<T>	no order, unique — O(1) avg

```
set<int> s;
s.insert(x);                    // O(log n)
s.erase(x);                     // erase by VALUE, O(log n)
s.find(x);                      // returns iterator or s.end()
s.count(x);                     // 0 or 1 (set) / freq (multiset)
auto it = s.lower_bound(x);     // first elem >= x
auto it2 = s.upper_bound(x);    // first elem > x
s.erase(it);                    // erase by iterator — 1 copy only!
```

> multiset: `s.erase(x)` deletes **all** copies of x. To delete one occurrence: `s.erase(s.find(x))`.

8. Map family

key-value

Container	Property
map<K,V>	sorted by key, unique keys
multimap<K,V>	sorted, duplicate keys OK
unordered_map<K,V>	no order, unique — O(1) avg

```
map<string,int> m;
m[key] = value;                 // insert/update – O(log n)
m.insert({key, value});
m.erase(key);
if (m.find(key) != m.end()) ..  // check existence (safe)
if (m.count(key)) ..            // also safe (no auto-insert)
for (auto &p : m) cout<<p.first<<p.second; // sorted by key
```

Careful: `m[key]` auto-inserts key with value 0 if absent — avoid for existence checks; use `find` / `count`.

9. Iterators

Type	Meaning
begin()/end()	forward, end() = past-last
rbegin()/rend()	reverse traversal
***it**	dereference to get value
next(it) / prev(it)	move iterator by 1

10. Common Algorithms

<algorithm>

```
sort(v.begin(), v.end());
reverse(v.begin(), v.end());
*max_element(v.begin(), v.end());
*min_element(v.begin(), v.end());
accumulate(v.begin(), v.end(), 0);      // sum, needs <numeric>
count(v.begin(), v.end(), x);           // freq of x
find(v.begin(), v.end(), x);            // iterator or end()
binary_search(v.begin(), v.end(), x);   // bool, needs sorted
lower_bound(v.begin(), v.end(), x);     // first >= x (sorted)
upper_bound(v.begin(), v.end(), x);     // first > x (sorted)
next_permutation(v.begin(), v.end());   // next lexico permutation
__builtin_popcount(x);                  // count set bits (int)
__gcd(a, b);                            // gcd of two numbers
swap(a, b);
```

11. Time Complexity Cheatsheet

Op	vector	set/map	u_set/u_map
insert	O(1)*	O(log n)	O(1) avg
delete	O(n)	O(log n)	O(1) avg
search	O(n)	O(log n)	O(1) avg
access	O(1)	—	—

*push_back amortized. sort() = O(n log n). unordered_* worst case O(n) (hash collisions) — avoid as map/set key if collisions matter (e.g. use pair hashing carefully).

12. Quick Picks for DSA

- Need order + dedupe → **set**
- Need frequency count → **map** / **unordered_map**
- Need fast O(1) lookup, order doesn't matter → **unordered_set/map**
- Need kth largest/smallest fast → **priority_queue**
- Need LIFO / matching brackets → **stack**
- Need FIFO / BFS → **queue**
- Need both-end insert/delete → **deque**
- Sliding window max → **deque** (monotonic)

In interviews/CP, prefer `unordered_map` / `unordered_set` for speed, but switch to `map` / `set` if you need sorted order or worst-case guarantees.

13. Handy Syntax

```
auto x = v.begin();         // type inference
for (auto &x : v) x *= 2;   // modify in place
string s = "abc"; s[0];     // string = char array
to_string(123); stoi("45"); stoll("45");
tuple<int,int,int> t = {1,2,3};
auto [a,b,c] = t;           // structured bindings (C++17)
memset(arr, 0, sizeof(arr));
fill(v.begin(), v.end(), 0);
```